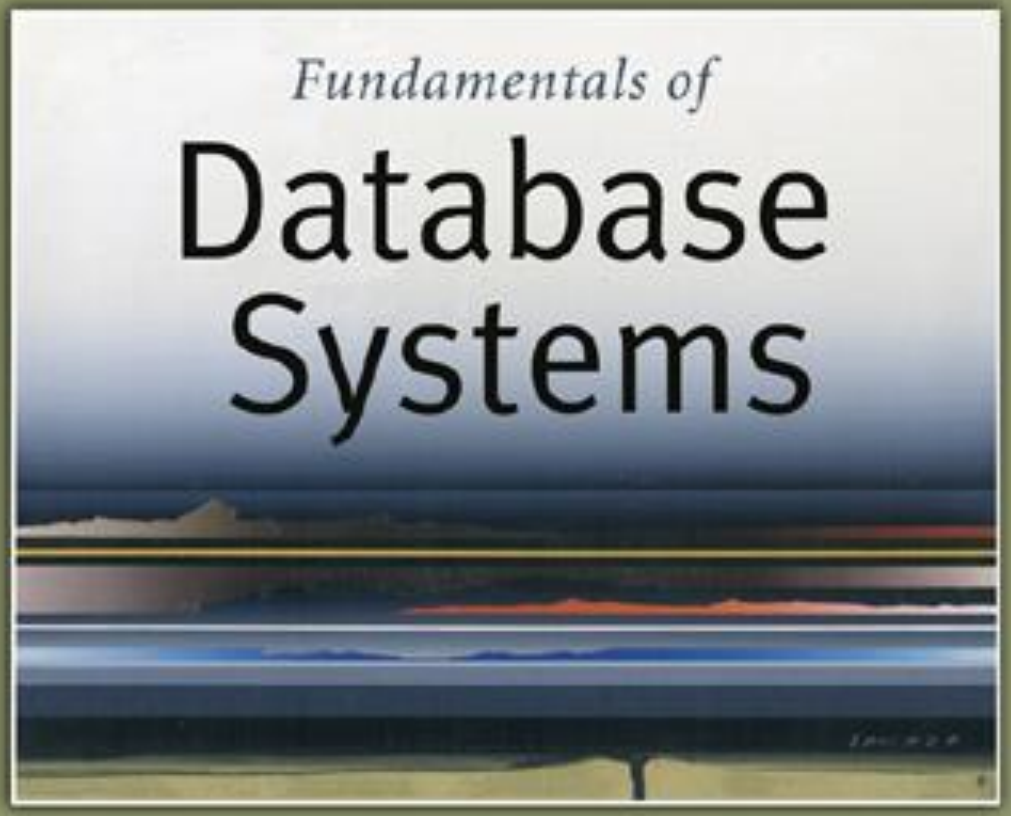
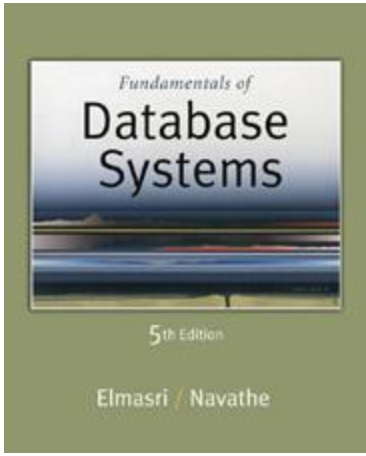


5th Edition

Elmasri / Navathe

Chapter 10

Functional Dependencies and Normalization for Relational Databases



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Chapter Outline

- 1 Informal Design Guidelines for Relational Databases
  - 1.1Semantics of the Relation Attributes
  - 1.2 Redundant Information in Tuples and Update Anomalies
  - 1.3 Null Values in Tuples
  - 1.4 Spurious Tuples
- 2 Functional Dependencies (FDs)
  - 2.1 Definition of FD
  - 2.2 Inference Rules for FDs
  - 2.3 Equivalence of Sets of FDs
  - 2.4 Minimal Sets of FDs

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Chapter Outline

- 3 Normal Forms Based on Primary Keys
  - 3.1 Normalization of Relations
  - 3.2 Practical Use of Normal Forms
  - 3.3 Definitions of Keys and Attributes Participating in Keys
  - 3.4 First Normal Form
  - 3.5 Second Normal Form
  - 3.6 Third Normal Form
- 4 General Normal Form Definitions (For Multiple Keys)
- 5 BCNF (Boyce-Codd Normal Form)

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

1 Informal Design Guidelines for Relational Databases (1)

- What is relational database design?
  - The grouping of attributes to form "good" relation schemas
- Two levels of relation schemas
  - The logical "user view" level
  - The storage "base relation" level
- Design is concerned mainly with base relations
- What are the criteria for "good" base relations?

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Informal Design Guidelines for Relational Databases (2)

- We first discuss informal guidelines for good relational design
- Then we discuss formal concepts of functional dependencies and normal forms
  - - 1NF (First Normal Form)
  - - 2NF (Second Normal Form)
  - - 3NF (Third Normal Form)
  - - BCNF (Boyce-Codd Normal Form)
- Additional types of dependencies, further normal forms, relational design algorithms by synthesis are discussed in Chapter 11

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

1.1 Semantics of the Relation Attributes

- GUIDELINE 1: Informally, each tuple in a relation should represent one entity or relationship instance. (Applies to individual relations and their attributes).
  - Attributes of different entities (EMPLOYEEs, DEPARTMENTs, PROJECTs) should not be mixed in the same relation
  - Only foreign keys should be used to refer to other entities
  - Entity and relationship attributes should be kept apart as much as possible.
- Bottom Line: *Design a schema that can be explained easily relation by relation. The semantics of attributes should be easy to interpret.*

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Figure 10.1 A simplified COMPANY relational database schema

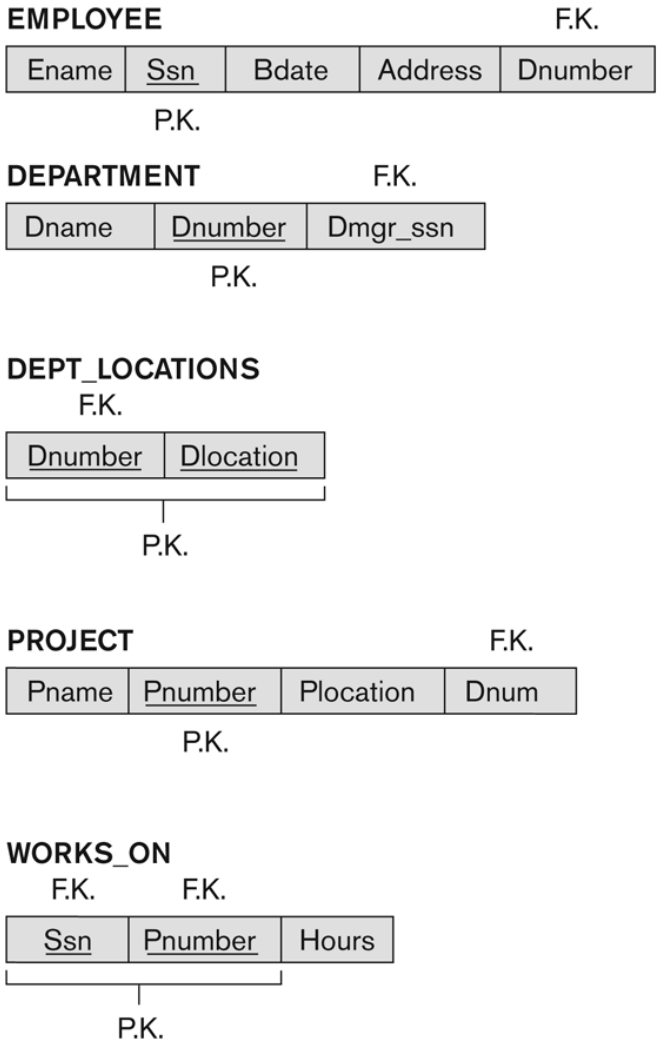


Figure 10.1
A simplified COMPANY relational database schema.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

1.2 Redundant Information in Tuples and Update Anomalies

- Information is stored redundantly
  - Wastes storage
  - Causes problems with update anomalies
    - Insertion anomalies
    - Deletion anomalies
    - Modification anomalies

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

EXAMPLE OF AN UPDATE ANOMALY

- Consider the relation:
  - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Update Anomaly:
  - Changing the name of project number P1 from “Billing” to “Customer-Accounting” may cause this update to be made for all 100 employees working on project P1.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

EXAMPLE OF AN INSERT ANOMALY

- Consider the relation:
  - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Insert Anomaly:
  - Cannot insert a project unless an employee is assigned to it.
- Conversely
  - Cannot insert an employee unless an he/she is assigned to a project.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

EXAMPLE OF AN DELETE ANOMALY

- Consider the relation:
  - EMP_PROJ(Emp#, Proj#, Ename, Pname, No_hours)
- Delete Anomaly:
  - When a project is deleted, it will result in deleting all the employees who work on that project.
  - Alternately, if an employee is the sole employee on a project, deleting that employee would result in deleting the corresponding project.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

StudentID	StudentName	CourseID	CourseName	InstructorName
1	John Doe	C101	Math	Dr. Smith
2	Jane Doe	C102	Physics	Dr. Johnson
3	Mark	C101	Math	Dr. Smith

1. Insert Anomaly

When adding new data to a database causes redundant or incomplete data.

•**Example**: Suppose you want to insert a new course (C103 - Chemistry) before any student enrolls in it. Since StudentID is part of the table, you can't add the course without associating it with a student.

Problem: Inserting course information without students forces you to leave other columns (e.g., StudentID, StudentName) null, or you need to insert dummy data, leading to inconsistencies.

Normalized Solution: Create a separate Course table to store course information independently of student data.

2. Update Anomaly
Occurs when updating data in a database requires changes in multiple rows, leading to inconsistent data.

Example: If the instructor for the Math course (C101) changes from "Dr. Smith" to "Dr. Anderson," you must update the InstructorName for every student enrolled in Math. In a large database, missing even one update could lead to inconsistent instructor data.

Problem: Failing to update all rows leads to inconsistent or outdated information about the instructor.

Normalized Solution: Split the table into separate Course and Instructor tables, where each course has only one associated instructor. This way, updating the instructor requires only one change in the Instructor table.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3. Deletion Anomaly
Occurs when deleting data unintentionally removes other valuable data.

Example: Suppose Mark, who is enrolled in the Math course, drops out, and his record is deleted from the StudentCourse table. If Mark was the only student taking Math, deleting his record will also delete the information about the Math course and its instructor.

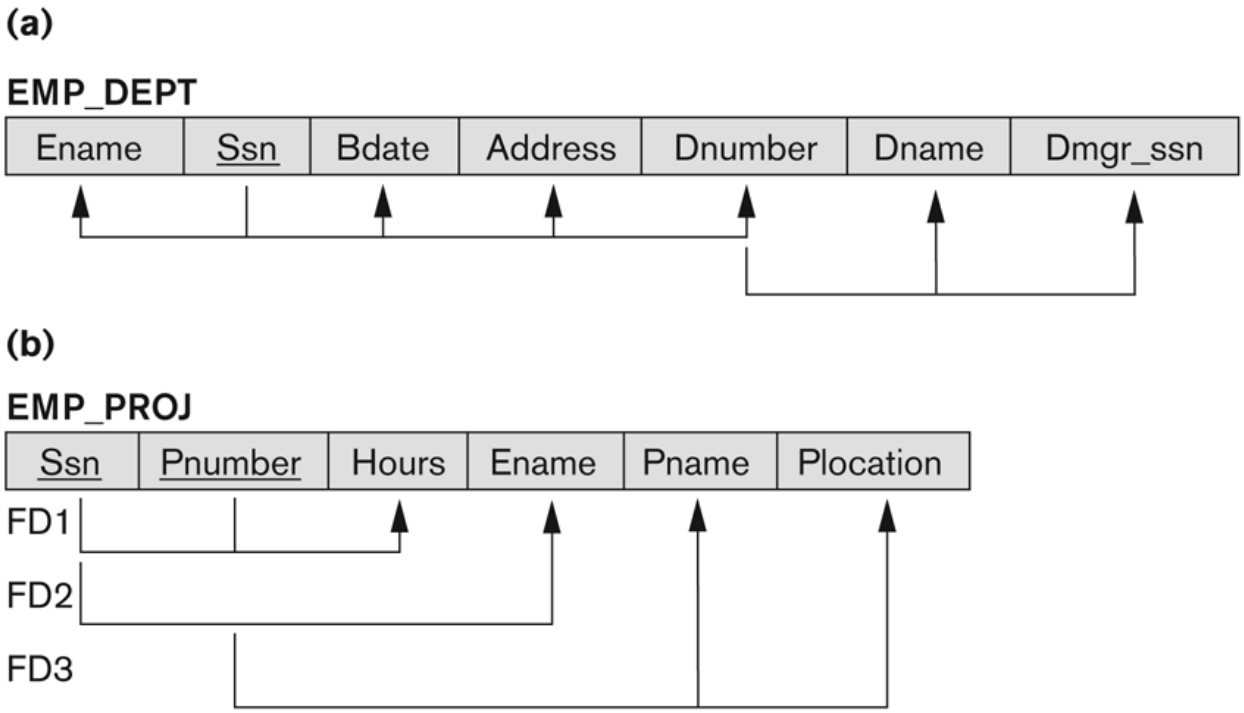
Problem: Deleting a student removes critical information about the course and the instructor, even though the course still exists.

Normalized Solution: Store Course and Instructor information in separate tables so that deleting a student doesn't affect the course or instructor information.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Figure 10.3 Two relation schemas suffering from update anomalies

Figure 10.3
Two relation schemas suffering from update anomalies.
(a) EMP_DEPT and
(b) EMP_PROJ.



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Figure 10.4 Example States for EMP_DEPT and EMP_PROJ

Figure 10.4
Example states for EMP_DEPT and EMP_PROJ resulting from applying NATURAL JOIN to the relations in Figure 10.2. These may be stored as base relations for performance reasons.

Redundancy

EMP_DEPT

Ename	Ssn	Bdate	Address	Dnumber	Dname	Dmgr_ssn
Smith, John B.	123456789	1965-01-09	731 Fondren, Houston, TX	5	Research	333445555
Wong, Franklin T.	333445555	1955-12-08	638 Voss, Houston, TX	5	Research	333445555
Zelaya, Alicia J.	999887777	1968-07-19	3321 Castle, Spring, TX	4	Administration	987654321
Wallace, Jennifer S.	987654321	1941-06-20	291 Berry, Bellaire, TX	4	Administration	987654321
Narayan, Ramesh K.	666884444	1962-09-15	975 FireOak, Humble, TX	5	Research	333445555
English, Joyce A.	453453453	1972-07-31	5631 Rice, Houston, TX	5	Research	333445555
Jabbar, Ahmad V.	987987987	1969-03-29	980 Dallas, Houston, TX	4	Administration	987654321
Borg, James E.	888665555	1937-11-10	450 Stone, Houston, TX	1	Headquarters	888665555

Redundancy Redundancy

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation
123456789	1	32.5	Smith, John B.	ProductX	Bellaire
123456789	2	7.5	Smith, John B.	ProductY	Sugarland
666884444	3	40.0	Narayan, Ramesh K.	ProductZ	Houston
453453453	1	20.0	English, Joyce A.	ProductX	Bellaire
453453453	2	20.0	English, Joyce A.	ProductY	Sugarland
333445555	2	10.0	Wong, Franklin T.	ProductY	Sugarland
333445555	3	10.0	Wong, Franklin T.	ProductZ	Houston
333445555	10	10.0	Wong, Franklin T.	Computerization	Stafford
333445555	20	10.0	Wong, Franklin T.	Reorganization	Houston
999887777	30	30.0	Zelaya, Alicia J.	Newbenefits	Stafford
999887777	10	10.0	Zelaya, Alicia J.	Computerization	Stafford
987987987	10	35.0	Jabbar, Ahmad V.	Computerization	Stafford
987987987	30	5.0	Jabbar, Ahmad V.	Newbenefits	Stafford
987654321	30	20.0	Wallace, Jennifer S.	Newbenefits	Stafford
987654321	20	15.0	Wallace, Jennifer S.	Reorganization	Houston
888665555	20	Null	Borg, James E.	Reorganization	Houston

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Guideline to Redundant Information in Tuples and Update Anomalies

- GUIDELINE 2:
  - Design a schema that does not suffer from the insertion, deletion and update anomalies.
  - If there are any anomalies present, then note them so that applications can be made to take them into account.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

1.3 Null Values in Tuples

- GUIDELINE 3:
  - Relations should be designed such that their tuples will have as few NULL values as possible
  - Attributes that are NULL frequently could be placed in separate relations (with the primary key)
- Reasons for nulls:
  - Attribute not applicable or invalid
  - Attribute value unknown (may exist)
  - Value known to exist, but unavailable

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

1.4 Spurious Tuples

- Bad designs for a relational database may result in erroneous results for certain JOIN operations
- The "lossless join" property is used to guarantee meaningful results for join operations

- GUIDELINE 4:
  - The relations should be designed to satisfy the lossless join condition.
  - No spurious tuples should be generated by doing a natural-join of any relations.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Example

Consider a relation `EmployeeProject` that records the relationship between employees and projects.
Original `EmployeeProject` Table:

EmployeeID	EmployeeName	ProjectID	ProjectName
1	Alice	101	AI System
2	Bob	102	E-Commerce
3	Charlie	101	AI System
4	Eve	103	Banking App

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Decomposition

Decompose this table into two relations:

1. Employee (containing `EmployeeID` and `EmployeeName`):

EmployeeID	EmployeeName
1	Alice
2	Bob
3	Charlie
4	Eve

2. Project (containing `ProjectID` and `ProjectName`):

ProjectID	ProjectName
101	AI System
102	E-Commerce
103	Banking App

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Joining the Decomposed Relations

Notice that this result contains many incorrect combinations of employees and projects, which were not part of the original `EmployeeProject` table. These are spurious tuples caused by a poor join between the decomposed tables.

Spurious Result:

EmployeeID	EmployeeName	ProjectID	ProjectName
1	Alice	101	AI System
1	Alice	102	E-Commerce
1	Alice	103	Banking App
2	Bob	101	AI System
2	Bob	102	E-Commerce
2	Bob	103	Banking App
3	Charlie	101	AI System
3	Charlie	102	E-Commerce
3	Charlie	103	Banking App
4	Eve	101	AI System
4	Eve	102	E-Commerce
4	Eve	103	Banking App

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Spurious Tuples (2)

- There are two important properties of decompositions:
  a) Non-additive or losslessness of the corresponding join
  b) Preservation of the functional dependencies.

- Note that:
  - Property (a) is extremely important and *cannot* be sacrificed.
  - Property (b) is less stringent and may be sacrificed. (See Chapter 11).

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Therefore, the **lossless join** property ensures that when the relations are joined back using correct keys (like EmployeeID and ProjectID), the original relation is reconstructed perfectly, with no extra or missing tuples.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

2.1 Functional Dependencies (1)

- Functional dependencies (FDs)
  - Are used to specify *formal measures* of the "goodness" of relational designs
  - And keys are used to define **normal forms** for relations
  - Are **constraints** that are derived from the *meaning* and *interrelationships* of the data attributes
- A set of attributes X *functionally determines* a set of attributes Y if the value of X determines a unique value for Y

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Functional Dependencies (2)

- In a relational database, functional dependency refers to the relationship between two sets of attributes within a relation. It is denoted as: X → Y
- X -> Y holds if whenever two tuples have the same value for X, they *must have* the same value for Y
  - For any two tuples t1 and t2 in any relation instance r(R): If t1[X]=t2[X], *then* t1[Y]=t2[Y]
- X -> Y in R specifies a *constraint* on all relation instances r(R)
- Written as X -> Y; can be displayed graphically on a relation schema as in Figures. (denoted by the arrow:).
- FDs are derived from the real-world constraints on the attributes

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Functional Dependencies

- **Explanation:**
- In simpler terms, if you know the value of **X**, you can determine the value of **Y**. If two tuples (rows) in a relation have the same value for **X**, then they must also have the same value for **Y**. This ensures that **Y** is uniquely determined by **X**.
- **Example:** Consider a table called **Student** with the following attributes:

StudentID	StudentName	DateOfBirth
101	John	15-05-2000
102	Alice	20-11-1999
103	Bob	30-07-2001

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Functional Dependencies

- **Explanation of the line:**
- **"A set of attributes X functionally determines a set of attributes Y if the value of X determines a unique value for Y"** means:
  - In the example above, **X = StudentID** and **Y = StudentName**.
  - For each value of **StudentID**, there is only one corresponding value for **StudentName**.
  - If two records have the same **StudentID**, they must have the same **StudentName**, so knowing the **StudentID** is sufficient to uniquely identify the **StudentName**.
- This relationship helps maintain the integrity and consistency of the database.
- This ensures that there are no anomalies like duplicate data or incorrect associations.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Examples of FD constraints (1)

- Social security number determines employee name
  - SSN -> ENAME
- Project number determines project name and location
  - PNUMBER -> {PNAME, PLOCATION}
- Employee ssn and project number determines the hours per week that the employee works on the project
  - {SSN, PNUMBER} -> HOURS

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Examples of FD constraints (2)

- An FD is a property of the attributes in the schema R
- The constraint must hold on *every* relation instance r(R)
- If K is a key of R, then K functionally determines all attributes in R
  - (since we never have two distinct tuples with t1[K]=t2[K])

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Examples of FD constraints

EmployeeID (Key)	Name	Salary	Department
101	Alice	60000	HR
102	Bob	70000	IT
103	Carol	65000	HR

- EmployeeID is the key of the relation Employee.
- Because EmployeeID uniquely identifies each row, knowing the EmployeeID allows us to determine the values of all other attributes (i.e., Name, Salary, and Department) for that row.
- For example:
- EmployeeID = 101 determines the values:
  - Name = Alice,
  - Salary = 60000,
  - Department = HR.
- There cannot be another tuple in the relation with the same EmployeeID but different values for Name, Salary, or Department.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

2.2 Inference Rules for FDs (1)

- Given a set of FDs F, we can **infer** additional FDs that hold whenever the FDs in F hold
- Armstrong's inference rules:
  - IR1. (**Reflexive**) If Y *subset-of* X, then X -> Y
  - IR2. (**Augmentation**) If X -> Y, then XZ -> YZ
    - (Notation: XZ stands for X U Z)
  - IR3. (**Transitive**) If X -> Y and Y -> Z, then X -> Z

- IR1, IR2, IR3 form a **sound** and **complete** set of inference rules
  - These are rules hold and all other rules that hold can be deduced from these

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Inference Rules for FDs (1)

Explanation of Armstrong's Inference Rules:

1.IR1: Reflexive Rule

If Y is a subset of X, then X → Y.

1. *This rule means that any set of attributes **X** can functionally determine any subset of itself. This is because the values of **X** trivially determine the values of the attributes that are part of **X** itself.*

***2.Example**:*

*If **X = {EmployeeID, Name}**, then **X → EmployeeID** and **X → Name** because both **EmployeeID** and **Name** are parts of **X**.*

3.IR2: Augmentation Rule

If X → Y, then XZ → YZ.

*(Where **XZ** is the union of **X** and **Z**)*

1. *This rule says that if **X** functionally determines **Y**, then adding the same set of attributes **Z** to both sides of the dependency will still hold. Essentially, you can augment both the left-hand side and the right-hand side with any additional attributes.*

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Example:
If EmployeeID → Name, then {EmployeeID, Department} → {Name, Department}. This is because if we know EmployeeID determines Name, adding the attribute Department on both sides doesn't affect the dependency.

•IR3: Transitive Rule

If X → Y and Y → Z, then X → Z.

•This rule states that if X determines Y and Y determines Z, then X also determines Z. This reflects the idea of transitivity in logical reasoning.

•Example:
If EmployeeID → DepartmentID and DepartmentID → DepartmentName, then by the transitive rule, EmployeeID → DepartmentName.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Why Armstrong's Rules Are Sound and Complete:

- ***Soundness*** *means that these rules are valid, i.e., they correctly derive functional dependencies from a set of given FDs.*

- ***Completeness*** *means that these rules are enough to derive all possible valid FDs from a set of given FDs. If any FD holds between attributes in a relation, you can deduce it using Armstrong's rules.*

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Example: Use of Armstrong's Rules:

Let's say you have the following FDs:
1.EmployeeID → DepartmentID
2.DepartmentID → Manager

*Using Armstrong's **transitive rule** (IR3):*
•*From **EmployeeID → DepartmentID** and **DepartmentID → Manager**, you can deduce that **EmployeeID → Manager**.*

*Using **augmentation** (IR2):*
•*If you know **EmployeeID → DepartmentID**, you can augment it by adding another attribute, say **{EmployeeID, Phone} → {DepartmentID, Phone}**.*

*Using **reflexive** (IR1):*
•*For any set, say **EmployeeID, DepartmentID**, you know that **{EmployeeID, DepartmentID} → {EmployeeID}**.*

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Inference Rules for FDs (2)

- Some additional inference rules that are useful:
  - **Decomposition:** If X -> YZ, then X -> Y and X -> Z
  - **Union:** If X -> Y and X -> Z, then X -> YZ
  - **Psuedotransitivity:** If X -> Y and WY -> Z, then WX -> Z

- The last three inference rules, as well as any other inference rules, can be deduced from IR1, IR2, and IR3 (completeness property)

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Decomposition Rule

- **Rule**:
  If X -> YZ, then X -> Y and X -> Z
- **Explanation**:
  If a set of attributes **X** determines a combined set of attributes **YZ**, then **X** must individually determine **Y** and **Z** as well. Essentially, you can decompose a functional dependency involving a set of attributes into two or more smaller functional dependencies.
- **Example**: **EmployeeID → {Name, DepartmentID}**
- According to the decomposition rule:
  - **EmployeeID → Name**
  - **EmployeeID → DepartmentID**
- This means if an employee's ID determines both the name and the department, then it must independently determine each of these as well.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Union Rule

- **Rule**:
  If X -> Y and X -> Z, then X -> YZ
- **Explanation**:
  If **X** determines **Y** and **X** also determines **Z**, then we can "union" these dependencies into one: **X** determines both **Y** and **Z** together, i.e., **YZ**. Essentially, you can combine separate functional dependencies into a single functional dependency.
- **Example**:
  EmployeeID → Name
  EmployeeID → DepartmentID
- According to the union rule:
  - **EmployeeID → {Name, DepartmentID}**
- This means if an employee's ID determines both the name and the department individually, it also determines the combination of these two.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Pseudotransitivity Rule

- **Rule**: If X -> Y and WY -> Z, then WX -> Z
- **Explanation**:
  This rule is similar to the transitivity rule but involves combining two dependencies. If **X** determines **Y**, and **WY** (where **W** is another set of attributes) determines **Z**, then **WX** together will determine **Z**. Essentially, you can "transfer" the dependency via **Y**, taking **W** into account.
- **Example**:
  EmployeeID → DepartmentID
  - **{Location, DepartmentID} → Manager**
- According to the pseudotransitivity rule:
  {Location, EmployeeID} → Manager
- This means if **EmployeeID** determines the department, and the combination of **Location** and **DepartmentID** determines the manager, then **Location** and **EmployeeID** together determine the manager.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Inference Rules for FDs (2)

- Decomposition Rule helps to break down complex dependencies into simpler ones, making the relationships easier to analyze.

- Union Rule allows you to combine dependencies to reflect more complex relationships.

- Pseudotransitivity Rule lets you deduce functional dependencies indirectly by considering multiple relationships.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Inference Rules for FDs (3)

- The **closure of a set of attributes** in a relation refers to the set of all attributes that can be functionally determined by that set of attributes using a given set of functional dependencies.
- In other words, for a set of attributes X, the **closure** X^+ is the set of all attributes that can be derived from X based on the functional dependencies in a relation.

- **Closure** of a set F of FDs is the set F^+ of all FDs that can be inferred from F

- **Closure** of a set of attributes X with respect to F is the set X^+ of all attributes that are functionally determined by X

- X^+ can be calculated by repeatedly applying IR1, IR2, IR3 using the FDs in F

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Why is Closure Important?

- **Determining Candidate Keys**: The closure helps in determining whether a set of attributes is a **superkey** or **candidate key**. If the closure of a set of attributes contains all the attributes in the relation, then it's a superkey.
- **Normalization**: Closure helps in determining whether a functional dependency holds or if certain attributes functionally determine others.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

2.3 Equivalence of Sets of FDs

- Two sets of FDs F and G are **equivalent** if:
  - Every FD in F can be inferred from G, and
  - Every FD in G can be inferred from F
  - Hence, F and G are equivalent if $F^+ = G^+$
- Definition (**Covers**):
  - F **covers** G if every FD in G can be inferred from F
    - (i.e., if G^+ *subset-of* F^+)
- F and G are equivalent if F covers G and G covers F
- There is an algorithm for checking equivalence of sets of FDs

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

2.4 Minimal Sets of FDs (1)

- A set of FDs is **minimal** if it satisfies the following conditions:
  1. Every dependency in F has a single attribute for its RHS.
  2. We cannot remove any dependency from F and have a set of dependencies that is equivalent to F.
  3. We cannot replace any dependency X -> A in F with a dependency Y -> A, where Y proper-subset-of X (Y subset-of X) and still have a set of dependencies that is equivalent to F.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Minimal Sets of FDs (2)

- Every set of FDs has an equivalent minimal set
- There can be several equivalent minimal sets
- There is no simple algorithm for computing a minimal set of FDs that is equivalent to a set F of FDs
- To synthesize a set of relations, we assume that we start with a set of dependencies that is a minimal set
  - E.g., see algorithms 11.2 and 11.4

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3 Normal Forms Based on Primary Keys

- 3.1 Normalization of Relations
- 3.2 Practical Use of Normal Forms
- 3.3 Definitions of Keys and Attributes Participating in Keys
- 3.4 First Normal Form
- 3.5 Second Normal Form
- 3.6 Third Normal Form

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3.1 Normalization of Relations (1)

- **Normalization:**
  - The process of decomposing unsatisfactory "bad" relations by breaking up their attributes into smaller relations

- **Normal form:**
  - Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Normalization of Relations (2)

- 2NF, 3NF, BCNF
  - based on keys and FDs of a relation schema
- 4NF
  - based on keys, multi-valued dependencies : MVDs; 5NF based on keys, join dependencies : JDs (Chapter 11)
- Additional properties may be needed to ensure a good relational design (lossless join, dependency preservation; Chapter 11)

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3.2 Practical Use of Normal Forms

- **Normalization** is carried out in practice so that the resulting designs are of high quality and meet the desirable properties
- The practical utility of these normal forms becomes questionable when the constraints on which they are based are *hard to understand* or to *detect*
- The database designers *need not* normalize to the highest possible normal form
  - (usually up to 3NF, BCNF or 4NF)
- **Denormalization**:
  - The process of storing the join of higher normal form relations as a base relation—which is in a lower normal form

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3.3 Definitions of Keys and Attributes Participating in Keys (1)

- A **superkey** of a relation schema R = {A1, A2, ..., An} is a set of attributes S *subset-of* R with the property that no two tuples t1 and t2 in any legal relation state r of R will have t1[S] = t2[S]

- A **key** K is a **superkey** with the *additional property* that removal of any attribute from K will cause K not to be a superkey any more.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Definitions of Keys and Attributes Participating in Keys (2)

- If a relation schema has more than one key, each is called a **candidate** key.
  - One of the candidate keys is *arbitrarily* designated to be the **primary key**, and the others are called **secondary keys**.
- A **Prime attribute** must be a member of *some* candidate key
- A **Nonprime attribute** is not a prime attribute—that is, it is not a member of any candidate key.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3.2 First Normal Form

- Disallows
  - composite attributes
  - multivalued attributes
  - **nested relations**; attributes whose values for an *individual tuple* are non-atomic

- Considered to be part of the definition of relation

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Figure 10.8 Normalization into 1NF

(a)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations



(b)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Figure 10.8
Normalization into 1NF. (a) A relation schema that is not in 1NF. (b) Example state of relation DEPARTMENT. (c) 1NF version of the same relation with redundancy.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Figure 10.9 Normalization nested relations into 1NF

(a)

EMP_PROJ

		Projs	
Ssn	Ename	Pnumber	Hours

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, AliciaJ.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

EMP_PROJ1

Ssn	Ename

EMP_PROJ2

Ssn	Pnumber	Hours

Figure 10.9
Normalizing nested relations into 1NF. (a) Schema of the EMP_PROJ relation with a *nested relation* attribute PROJS. (b) Example extension of the EMP_PROJ relation showing nested relations within each tuple. (c) Decomposition of EMP_PROJ into relations EMP_PROJ1 and EMP_PROJ2 by propagating the primary key.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3.3 Second Normal Form (1)

- Uses the concepts of **FDs, primary key**
- Definitions
  - **Prime attribute:** An attribute that is member of the primary key K
  - **Full functional dependency:** a FD Y -> Z where removal of any attribute from Y means the FD does not hold any more
- Examples:
  - {SSN, PNUMBER} -> HOURS is a full FD since neither SSN -> HOURS nor PNUMBER -> HOURS hold
  - {SSN, PNUMBER} -> ENAME is not a full FD (it is called a partial dependency) since SSN -> ENAME also holds

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Second Normal Form (2)

- A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on the primary key

- R can be decomposed into 2NF relations via the process of 2NF normalization

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Figure 10.10 Normalizing into 2NF and 3NF

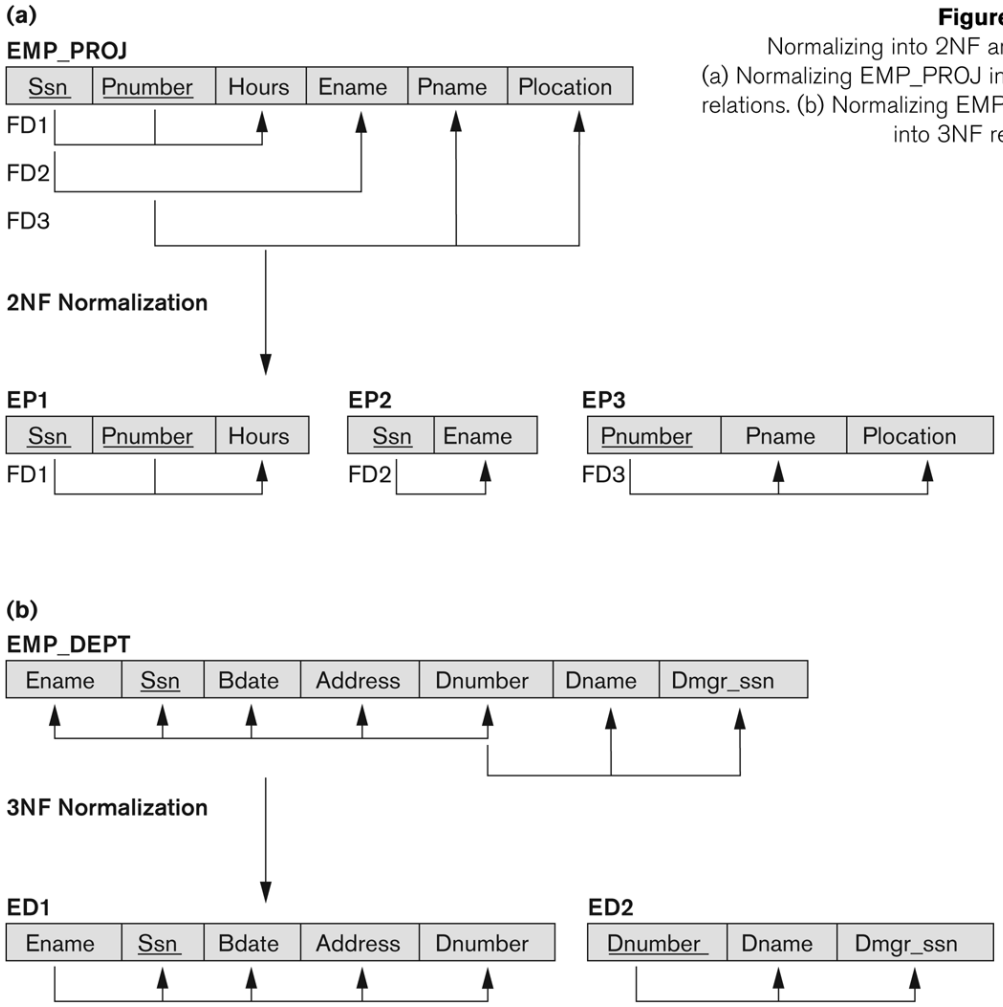


Figure 10.10
Normalizing into 2NF and 3NF. (a) Normalizing EMP_PROJ into 2NF relations. (b) Normalizing EMP_DEPT into 3NF relations.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Figure 10.11 Normalization into 2NF and 3NF

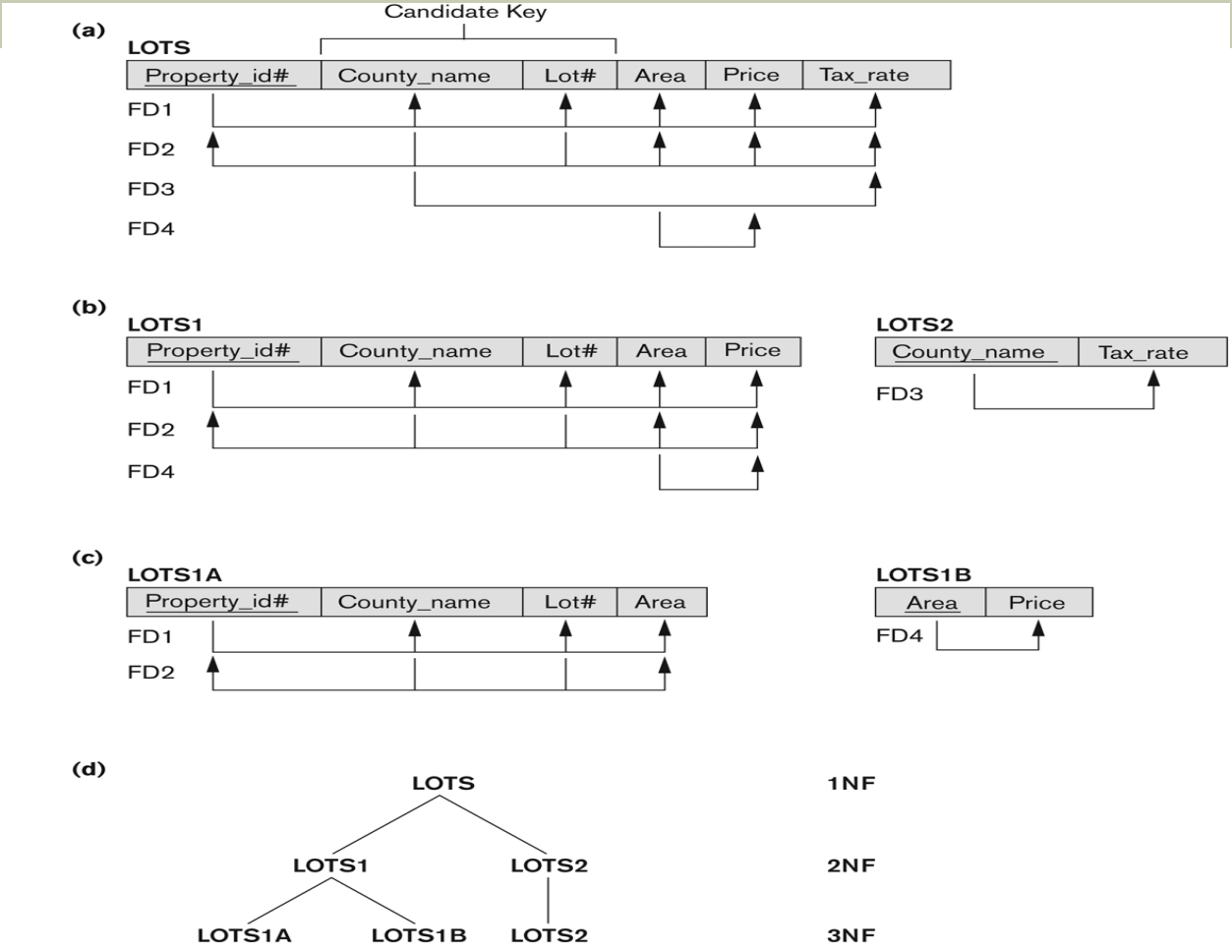


Figure 10.11
Normalization into 2NF and 3NF. (a) The LOTS relation with its functional dependencies FD1 through FD4. (b) Decomposing into the 2NF relations LOTS1 and LOTS2. (c) Decomposing LOTS1 into the 3NF relations LOTS1A and LOTS1B. (d) Summary of the progressive normalization of LOTS.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3.4 Third Normal Form (1)

- Definition:
  - **Transitive functional dependency:** a FD X -> Z that can be derived from two FDs X -> Y and Y -> Z
- Examples:
  - SSN -> DMGRSSN is a **transitive** FD
    - Since SSN -> DNUMBER and DNUMBER -> DMGRSSN hold
  - SSN -> ENAME is **non-transitive**
    - Since there is no set of attributes X where SSN -> X and X -> ENAME

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Third Normal Form (2)

- A relation schema R is in **third normal form (3NF)** if it is in 2NF *and* no non-prime attribute A in R is transitively dependent on the primary key
- R can be decomposed into 3NF relations via the process of 3NF normalization
- NOTE:
  - In X -> Y and Y -> Z, with X as the primary key, we consider this a problem only if Y is not a candidate key.
  - When Y is a candidate key, there is no problem with the transitive dependency .
  - E.g., Consider EMP (SSN, Emp#, Salary).
    - Here, SSN -> Emp# -> Salary and Emp# is a candidate key.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Transitive dependency

- A **transitive dependency** occurs when you have the following pattern of functional dependencies:

1. $X \rightarrow Y$
2. $Y \rightarrow Z$

- This means that X determines Y, and Y determines Z. As a result, X indirectly determines Z via Y.
- In database design, transitive dependencies can be problematic because they can introduce **redundancy** and **anomalies** (such as update, insertion, and deletion anomalies), which 3NF aims to eliminate.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Why Transitive Dependency is a Problem (and When It Isn't):

- A transitive dependency is considered problematic **only if Y is not a candidate key**. If Y is **not a candidate key**, this means that Y does not uniquely identify records, and the transitive dependency can lead to redundancy.

- However, if Y **is a candidate key**, the transitive dependency is **not a problem**. This is because candidate keys uniquely identify records in the table, so they are functionally independent, and no redundancy arises.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Problematic Scenario: EMP (SSN, Emp#, Salary)

•If Emp# is **not** a candidate key, this transitive dependency would be problematic, because Emp# would not uniquely identify records. Thus, the relationship between SSN and Salary would cause redundancy.

Non-Problematic Scenario:

•In this case, however, **Emp# is a candidate key**, meaning that Emp# uniquely identifies employees. Since Emp# is unique, no redundancy or anomaly is introduced, even though there is a transitive dependency. As a result, the transitive dependency does **not** violate 3NF.

Conclusion:

A transitive dependency (like X→Y→Z) is only considered a problem in database design if Y is **not** a candidate key. If Y **is** a candidate key, then there is no problem, as candidate keys uniquely identify records and prevent redundancy. In the example of EMP (SSN, Emp#, Salary), Emp# is a candidate key, so the transitive dependency does not violate the normal form.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Normal Forms Defined Informally

- 1st normal form
  - All attributes depend on **the key**
- 2nd normal form
  - All attributes depend on **the whole key**
- 3rd normal form
  - All attributes depend on **nothing but the key**

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

4 General Normal Form Definitions (For Multiple Keys) (1)

- The above definitions consider the primary key only
- The following more general definitions take into account relations with multiple candidate keys
- A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on *every* key of R

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

General Normal Form Definitions (2)

- Definition:
  - **Superkey** of relation schema R - a set of attributes S of R that contains a key of R
  - A relation schema R is in **third normal form (3NF)** if whenever a FD X -> A holds in R, then either:
    - (a) X is a superkey of R, or
    - (b) A is a prime attribute of R
- NOTE: Boyce-Codd normal form disallows condition (b) above

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

5 BCNF (Boyce-Codd Normal Form)

- A relation schema R is in **Boyce-Codd Normal Form (BCNF)** if whenever an **FD X -> A** holds in R, then **X is a superkey** of R
- Each normal form is strictly stronger than the previous one
  - Every 2NF relation is in 1NF
  - Every 3NF relation is in 2NF
  - Every BCNF relation is in 3NF
- There exist relations that are in 3NF but not in BCNF
- The goal is to have each relation in BCNF (or 3NF)

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Figure 10.12 Boyce-Codd normal form

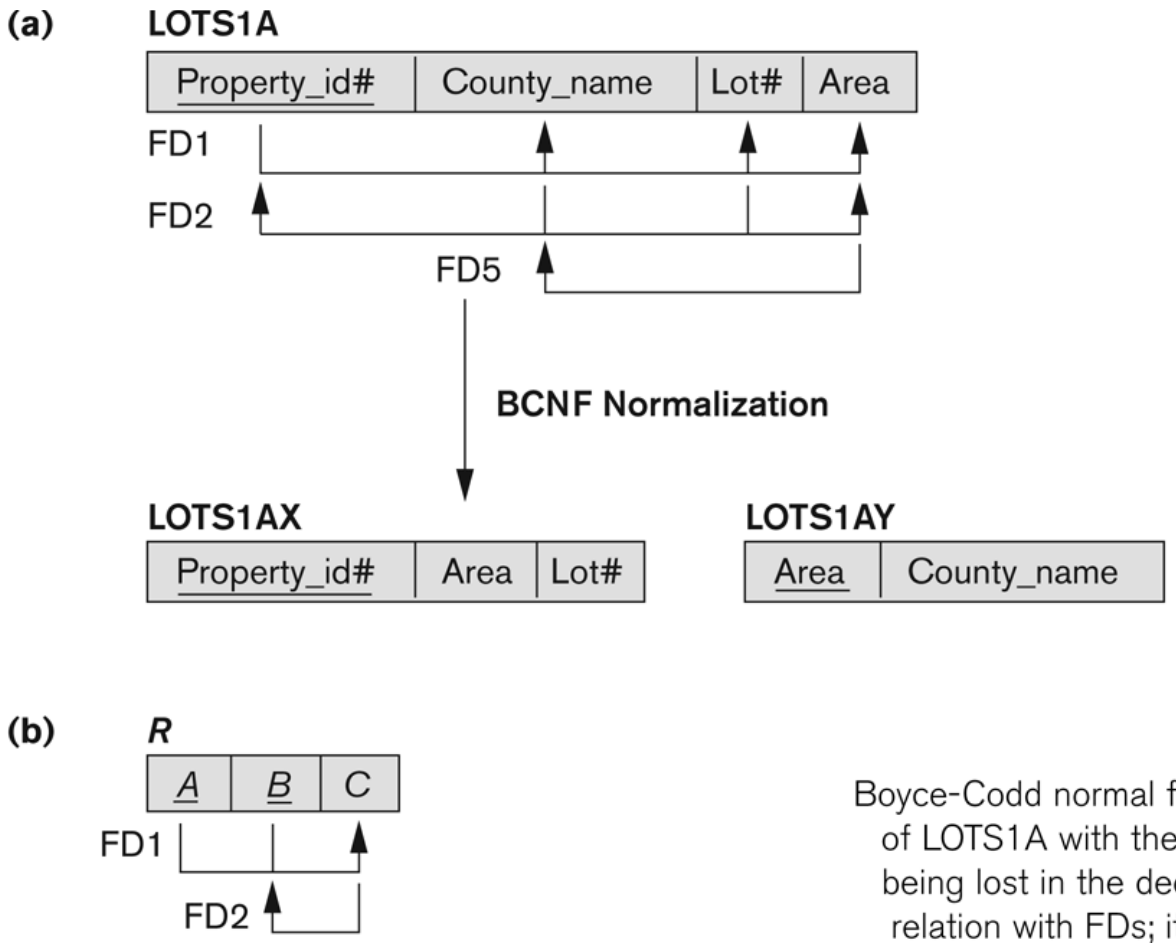


Figure 10.12
Boyce-Codd normal form. (a) BCNF normalization of LOTS1A with the functional dependency FD2 being lost in the decomposition. (b) A schematic relation with FDs; it is in 3NF, but not in BCNF.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Figure 10.13 a relation TEACH that is in 3NF but not in BCNF

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

Figure 10.13
A relation TEACH that is in 3NF but not BCNF.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Achieving the BCNF by Decomposition (1)

- Two FDs exist in the relation TEACH:
  - fd1: { student, course} -> instructor
  - fd2: instructor -> course
- {student, course} is a candidate key for this relation and that the dependencies shown follow the pattern in Figure 10.12 (b).
  - So this relation is in 3NF *but not in* BCNF
- A relation **NOT** in BCNF should be decomposed so as to meet this property, while possibly forgoing the preservation of all functional dependencies in the decomposed relations.
  - (See Algorithm 11.3)

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Achieving the BCNF by Decomposition (2)

- Three possible decompositions for relation TEACH
  - {student, instructor} and {student, course}
  - {course, instructor } and {course, student}
  - {instructor, course } and {instructor, student}
- All three decompositions will lose fd1.
  - We have to settle for sacrificing the functional dependency preservation. But we cannot sacrifice the non-additivity property after decomposition.
- Out of the above three, only the 3rd decomposition will not generate spurious tuples after join.(and hence has the non-additivity property).
- A test to determine whether a binary decomposition (decomposition into two relations) is non-additive (lossless) is discussed in section 11.1.4 under Property LJ1. Verify that the third decomposition above meets the property.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Key Goals:

- **Lossless Join (Non-additive property)**: This ensures that after decomposing the relation and then joining it back together, we do not generate spurious (incorrect or additional) tuples.
- **Functional Dependency Preservation**: This means that the functional dependencies of the original relation should be preserved in the decomposed relations.
- However, in this case, **we are sacrificing functional dependency preservation** but still want to ensure the non-additivity (lossless join) property.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

1. Decomposition 1: {student, instructor} and {student, course}

- **Common attribute**: student
- Does student form a **superkey** for either {student, instructor} or {student, course}?
  - No, student alone cannot uniquely determine instructor or course in these relations.

Therefore, this decomposition **does not satisfy** the lossless join property, and spurious tuples could be generated after the join.

2. Decomposition 2: {course, instructor} and {course, student}

- **Common attribute**: course
- Does course form a **superkey** for either {course, instructor} or {course, student}?
  - No, course alone cannot uniquely determine instructor or student.

Therefore, this decomposition **does not satisfy** the lossless join property either, and spurious tuples could result after the join.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3. Decomposition 3: {instructor, course} and {instructor, student}

- **Common attribute**: instructor
- Does instructor form a **superkey** for either {instructor, course} or {instructor, student}?
  - Yes, in this decomposition, instructor is a key in both relations because:
    - In {instructor, course}, an instructor teaches certain courses, so instructor can uniquely identify the course.
    - In {instructor, student}, the instructor is associated with specific students, so instructor can uniquely identify the student.

Therefore, this decomposition **satisfies** the lossless join (non-additive) property, meaning no spurious tuples will be generated after the join.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Summary:

1. **1NF**: No repeating groups or arrays, all values are atomic.
2. **2NF**: No partial dependencies; all non-key attributes depend on the full composite key.
3. **3NF**: No transitive dependencies; all non-key attributes depend only on the primary key.
4. **BCNF**: Every determinant (left side of functional dependency) is a superkey.

These steps progressively eliminate redundancy and ensure the database is organized efficiently.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Chapter Outline

- Informal Design Guidelines for Relational Databases
- Functional Dependencies (FDs)
  - Definition, Inference Rules, Equivalence of Sets of FDs, Minimal Sets of FDs
- Normal Forms Based on Primary Keys
- General Normal Form Definitions (For Multiple Keys)
- BCNF (Boyce-Codd Normal Form)

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe